

TRD

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Customer Order Analytics ETL

- Document Version: 1.0
- Date: 2024-01-15
- Prepared By: Senior AWS Data Architect
- Project: Customer Order Analytics ETL
- Related FRD Version: 1.0
-

1. DOCUMENT OVERVIEW

1.1 Purpose

This Technical Requirements Document (TRD) translates the Functional Requirements Document (FRD) into detailed technical specifications for implementing the Customer Order Analytics ETL pipeline using AWS Glue. This document provides implementation-ready technical design that enables direct code generation without ambiguity.

1.2 Scope

This TRD covers:

- AWS Glue job implementation specifications for PySpark/Scala
- Detailed source-to-target data mappings with exact schemas
- Technical transformation logic and algorithms
- Data quality validation implementation details
- Error handling and logging mechanisms
- Configuration management via YAML
- Unit and integration testing specifications
- AWS Glue job execution parameters and runtime configuration

This TRD does NOT cover:

- AWS infrastructure provisioning (VPC, subnets, security groups)
- IAM role and policy creation
- CloudWatch alarm configuration
- Cost optimization strategies
- Downstream consumer applications

1.3 Assumptions

1. AWS Glue 3.0 or higher with Python 3.7+ or Scala 2.12
2. Spark 3.1.1 or higher runtime environment
3. S3 bucket s3://adif-sdlc exists with appropriate read/write permissions
4. CSV files use UTF-8 encoding with comma delimiter
5. CSV files contain header row as first line
6. Glue job has IAM role with s3:GetObject on source paths and s3:PutObject on target paths
7. Glue Data Catalog is available for metadata management
8. Job will run in batch mode with daily schedule
9. DPU allocation: minimum 2 DPUs, recommended 5-10 DPUs for production
10. Maximum job timeout: 60 minutes
11. Spark configuration uses default settings unless specified
12. YAML configuration file will be stored in s3:///config/job_config.yaml
13. Error logs will be written to s3://adif-sdlc/logs/errors/
14. Job execution logs will be written to CloudWatch Logs
15. Timestamp format follows ISO 8601: YYYY-MM-DD HH:MM:SS
16. Proper case conversion uses Unicode-aware title case transformation
17. Window functions are supported in Spark SQL for ORDER_COUNT calculation
18. Parquet compression codec: SNAPPY (default)
19. No schema evolution during initial implementation phase
20. Test data files will be stored in s3://adif-sdlc/test_data/

•

2. TECHNICAL REQUIREMENTS

TR-EXTRACT-001: Extract Customer Data from S3

- Related Functional Requirement ID(s): FR-EXTRACT-001
- Objective:

Implement S3 CSV file reader to load customer master data into Spark DataFrame with schema enforcement and record count logging.

- Source Details:
- Source Type: S3 CSV File
- Source Path: s3://adif-sdlc/busi_req_doc/customers.csv
- File Format: CSV
- Delimiter: Comma (,)

- Header: Present (first row)
- Encoding: UTF-8
- Compression: None
- Target Details:
- Target Type: In-memory Spark DataFrame
- DataFrame Name: df_customers_raw
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Constraints	
	-----	-----	-----	-----	-----	
	CID	Integer	32-bit	NOT NULL	Primary Key, > 0	
	FNAME	String	Variable	NOT NULL	Non-empty	
	LNAME	String	Variable	NOT NULL	Non-empty	

- Output Schema:

Same as Input Schema (pass-through at this stage)

- Processing / Transformation Logic:
 1. Initialize Spark session with application name "CustomerOrderETL"
 2. Configure Spark to read CSV with following options:
 - `header=true`
 - `inferSchema=false` (explicit schema enforcement)
 - `delimiter=`,`
 - `encoding=UTF-8`
 - `mode=FAILFAST` (fail on malformed records)
 3. Define explicit schema using StructType:

```

```
StructType([
 StructField("CID", IntegerType(), False),
 StructField("FNAME", StringType(), False),
 StructField("LNAME", StringType(), False)
])
```

```

4. Read CSV file from s3://adif-sdlc/busi_req_doc/customers.csv
5. Load into DataFrame df_customers_raw
6. Execute count() action to retrieve record count

7. Log record count with INFO level: "Loaded {count} customer records from source"

8. Cache DataFrame if record count > 10,000 for performance optimization

- Validation Rules:

- File must exist at specified S3 path
- File must be readable and parsable as CSV
- Schema must match expected structure (3 columns with correct names and types)
- At least one data row must exist beyond header

- Error Handling and Logging:

- File Not Found: Log ERROR "Source file not found: s3://adif-sdlc/busi_req_doc/customers.csv" → Terminate job with exit code 1
- Empty File: Log WARN "Source file is empty, proceeding with zero records" → Continue with empty DataFrame
- Schema Mismatch: Log ERROR "Schema validation failed. Expected: CID(Int), FNAME(String), LNAME(String)" → Terminate job with exit code 2
- Malformed CSV: Log ERROR "CSV parsing failed at line {line_number}: {error_message}" → Terminate job with exit code 3
- S3 Access Denied: Log ERROR "Access denied to S3 path: {path}" → Terminate job with exit code 4

- Runtime Parameters:

- `source.customers.path`: s3://adif-sdlc/busi_req_doc/customers.csv (from YAML config)
- `source.customers.format`: csv (from YAML config)
- `spark.sql.files.maxPartitionBytes`: 128MB (default)

- Operational Notes:

- Expected file size: < 100MB for initial implementation
- Read operation should complete within 30 seconds for files < 100MB
- Monitor S3 request metrics for throttling
- Implement retry logic with exponential backoff for transient S3 errors (max 3 retries)
-

TR-EXTRACT-002: Extract Order Data from S3

- Related Functional Requirement ID(s): FR-EXTRACT-002

- Objective:

Implement S3 CSV file reader to load order transaction data into Spark DataFrame with schema enforcement and record count logging.

- Source Details:
- Source Type: S3 CSV File
- Source Path: s3://adif-sdlc/busi_req_doc/orders.csv
- File Format: CSV
- Delimiter: Comma (,)
- Header: Present (first row)
- Encoding: UTF-8
- Compression: None
- Target Details:
- Target Type: In-memory Spark DataFrame
- DataFrame Name: df_orders_raw
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Constraints	
	-----	-----	-----	-----	-----	
	OID	Integer	32-bit	NOT NULL	Primary Key, > 0	
	CID	Integer	32-bit	NOT NULL	Foreign Key, > 0	
	ONAME	String	Variable	NOT NULL	Non-empty	

- Output Schema:

Same as Input Schema (pass-through at this stage)

- Processing / Transformation Logic:
 1. Use existing Spark session from TR-EXTRACT-001
 2. Configure Spark to read CSV with following options:
 - `header=true`
 - `inferSchema=false` (explicit schema enforcement)
 - `delimiter=`,`
 - `encoding=UTF-8`
 - `mode=FAILFAST` (fail on malformed records)
 3. Define explicit schema using StructType:

...

```
StructType([
  StructField("OID", IntegerType(), False),
  StructField("CID", IntegerType(), False),
  StructField("ONAME", StringType(), False)
```

)

...

4. Read CSV file from s3://adif-sdlc/busi_req_doc/orders.csv
5. Load into DataFrame df_orders_raw
6. Execute count() action to retrieve record count
7. Log record count with INFO level: "Loaded {count} order records from source"
8. Cache DataFrame if record count > 10,000 for performance optimization

- Validation Rules:

- File must exist at specified S3 path
- File must be readable and parsable as CSV
- Schema must match expected structure (3 columns with correct names and types)
- At least one data row must exist beyond header
- Error Handling and Logging:
 - File Not Found: Log ERROR "Source file not found: s3://adif-sdlc/busi_req_doc/orders.csv" → Terminate job with exit code 1
 - Empty File: Log WARN "Source file is empty, proceeding with zero records" → Continue with empty DataFrame
 - Schema Mismatch: Log ERROR "Schema validation failed. Expected: OID(Int), CID(Int), ONAME(String)" → Terminate job with exit code 2
 - Malformed CSV: Log ERROR "CSV parsing failed at line {line_number}: {error_message}" → Terminate job with exit code 3
 - S3 Access Denied: Log ERROR "Access denied to S3 path: {path}" → Terminate job with exit code 4

- Runtime Parameters:

- `source.orders.path`: s3://adif-sdlc/busi_req_doc/orders.csv (from YAML config)
- `source.orders.format`: csv (from YAML config)
- `spark.sql.files.maxPartitionBytes`: 128MB (default)

- Operational Notes:

- Expected file size: < 500MB for initial implementation
- Read operation should complete within 60 seconds for files < 500MB
- Monitor S3 request metrics for throttling
- Implement retry logic with exponential backoff for transient S3 errors (max 3 retries)
-

TR-VALIDATE-001: Validate Customer Data Quality

- Related Functional Requirement ID(s): FR-VALIDATE-001

- Objective:

Implement comprehensive data quality validation for customer records including null checks, data type validation, and duplicate removal with detailed error logging.

- Source Details:

- Source Type: In-memory Spark DataFrame
- Source Name: df_customers_raw (from TR-EXTRACT-001)

- Target Details:

- Target Type: In-memory Spark DataFrame
- Target Name: df_customers_valid
- Error Log Target: s3://adif-sdlc/logs/errors/customers/run_date=

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CID	Integer	32-bit	NOT NULL	
	FNAME	String	Variable	NOT NULL	
	LNAME	String	Variable	NOT NULL	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Notes	
	-----	-----	-----	-----	-----	
	CID	Integer	32-bit	NOT NULL	Validated, unique	
	FNAME	String	Variable	NOT NULL	Validated, non-empty	
	LNAME	String	Variable	NOT NULL	Validated, non-empty	

- Processing / Transformation Logic:

1. Create Validation Flags:

```
```
```

Add columns using withColumn():

- is\_cid\_null: CID IS NULL
- is\_cid\_invalid: CID IS NOT NULL AND CID <= 0
- is\_fname\_null: FNAME IS NULL
- is\_fname\_empty: FNAME IS NOT NULL AND trim(FNAME) = ""
- is\_lname\_null: LNAME IS NULL

- is\_lname\_empty: LNAME IS NOT NULL AND trim(LNAME) = "
- is\_valid: NOT (is\_cid\_null OR is\_cid\_invalid OR is\_fname\_null OR is\_fname\_empty OR is\_lname\_null OR is\_lname\_empty)

...

## 2. Separate Valid and Invalid Records:

...

```
df_customers_invalid = df_customers_raw.filter(col("is_valid") == False)
df_customers_valid_temp = df_customers_raw.filter(col("is_valid") == True)
```

...

## 3. Add Rejection Reasons to Invalid Records:

...

Use when().otherwise() chain to create rejection\_reason column:

- WHEN is\_cid\_null THEN "CID is NULL"
- WHEN is\_cid\_invalid THEN "CID must be positive integer"
- WHEN is\_fname\_null THEN "FNAME is NULL"
- WHEN is\_fname\_empty THEN "FNAME is required"
- WHEN is\_lname\_null THEN "LNAME is NULL"
- WHEN is\_lname\_empty THEN "LNAME is required"

...

## 4. Remove Duplicates:

...

```
df_customers_valid = df_customers_valid_temp.dropDuplicates(["CID"])
```

...

## 5. Calculate Metrics:

...

```
total_records = df_customers_raw.count()
invalid_records = df_customers_invalid.count()
duplicate_records = df_customers_valid_temp.count() - df_customers_valid.count()
valid_records = df_customers_valid.count()
```

...

## 6. Log Validation Summary:

...

INFO: "Customer Validation Summary:"



```
INFO: " Total Records: {total_records}"
INFO: " Valid Records: {valid_records}"
INFO: " Invalid Records: {invalid_records}"
INFO: " Duplicate Records Removed: {duplicate_records}"
...

```

#### 7. Write Error Log:

```
...

If invalid_records > 0:
Write df_customers_invalid (with rejection_reason) to error log location
Format: CSV with header
Partition by: run_date
...

```

- Validation Rules:
  - Rule V1: CID must not be NULL
  - Rule V2: CID must be positive integer ( $> 0$ )
  - Rule V3: FNAME must not be NULL
  - Rule V4: FNAME must not be empty string after trimming whitespace
  - Rule V5: LNAME must not be NULL
  - Rule V6: LNAME must not be empty string after trimming whitespace
  - Rule V7: CID must be unique (first occurrence retained)
- Error Handling and Logging:
  - Validation Failure: Log WARN for each validation rule violation with count
  - Error Log Write Failure: Log ERROR "Failed to write error log to S3: {error}" → Continue job execution
  - Zero Valid Records: Log WARN "No valid customer records after validation" → Continue with empty DataFrame
  - High Rejection Rate: If  $\text{invalid\_records} / \text{total\_records} > 0.5$ , log WARN "High rejection rate: {percentage}%"
- Runtime Parameters:
  - ``validation.customers.error_log_path``: s3://adif-sdlc/logs/errors/customers/ (from YAML config)
  - ``validation.customers.duplicate_key``: CID (from YAML config)
- Operational Notes:
  - Validation should complete within 2 minutes for 10M records
  - Error log partitioning by run\_date enables efficient historical analysis

- Consider sampling error log if invalid\_records > 100,000 (write first 100,000 + summary)
- Drop validation flag columns before proceeding to next stage
- 

### ### TR-VALIDATE-002: Validate Order Data Quality

- Related Functional Requirement ID(s): FR-VALIDATE-002
- Objective:

Implement comprehensive data quality validation for order records including null checks, data type validation, and duplicate removal with detailed error logging.

- Source Details:
  - Source Type: In-memory Spark DataFrame
  - Source Name: df\_orders\_raw (from TR-EXTRACT-002)
- Target Details:
  - Target Type: In-memory Spark DataFrame
  - Target Name: df\_orders\_valid
  - Error Log Target: s3://adif-sdlc/logs/errors/orders/run\_date=
- Input Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | OID         | Integer   | 32-bit                 | NOT NULL |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL |  |
|  | ONAME       | String    | Variable               | NOT NULL |  |

- Output Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Notes                |  |
|--|-------------|-----------|------------------------|----------|----------------------|--|
|  | -----       | -----     | -----                  | -----    | -----                |  |
|  | OID         | Integer   | 32-bit                 | NOT NULL | Validated, unique    |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL | Validated, > 0       |  |
|  | ONAME       | String    | Variable               | NOT NULL | Validated, non-empty |  |

- Processing / Transformation Logic:

1. Create Validation Flags:

\*\*\*

Add columns using withColumn():

- is\_oid\_null: OID IS NULL
- is\_oid\_invalid: OID IS NOT NULL AND OID <= 0
- is\_cid\_null: CID IS NULL
- is\_cid\_invalid: CID IS NOT NULL AND CID <= 0
- is\_otype\_name\_null: OTYPE IS NULL
- is\_otype\_name\_empty: OTYPE IS NOT NULL AND trim(OTYPE) = ""
- is\_valid: NOT (is\_oid\_null OR is\_oid\_invalid OR is\_cid\_null OR is\_cid\_invalid OR is\_otype\_name\_null OR is\_otype\_name\_empty)

...

## 2. Separate Valid and Invalid Records:

...

```
df_orders_invalid = df_orders_raw.filter(col("is_valid") == False)
df_orders_valid_temp = df_orders_raw.filter(col("is_valid") == True)
```

...

## 3. Add Rejection Reasons to Invalid Records:

...

Use when().otherwise() chain to create rejection\_reason column:

- WHEN is\_oid\_null THEN "OID is NULL"
- WHEN is\_oid\_invalid THEN "OID must be positive integer"
- WHEN is\_cid\_null THEN "CID is NULL"
- WHEN is\_cid\_invalid THEN "CID must be positive integer"
- WHEN is\_otype\_name\_null THEN "OTYPE is NULL"
- WHEN is\_otype\_name\_empty THEN "OTYPE is required"

...

## 4. Remove Duplicates:

...

```
df_orders_valid = df_orders_valid_temp.dropDuplicates(["OID"])
```

...

## 5. Calculate Metrics:

...

```
total_records = df_orders_raw.count()
invalid_records = df_orders_invalid.count()
duplicate_records = df_orders_valid_temp.count() - df_orders_valid.count()
```

```
valid_records = df_orders_valid.count()
```

```
...
```

#### 6. Log Validation Summary:

```
...
```

```
INFO: "Order Validation Summary:"
```

```
INFO: " Total Records: {total_records}"
```

```
INFO: " Valid Records: {valid_records}"
```

```
INFO: " Invalid Records: {invalid_records}"
```

```
INFO: " Duplicate Records Removed: {duplicate_records}"
```

```
...
```

#### 7. Write Error Log:

```
...
```

```
If invalid_records > 0:
```

```
Write df_orders_invalid (with rejection_reason) to error log location
```

```
Format: CSV with header
```

```
Partition by: run_date
```

```
...
```

- Validation Rules:

- Rule V1: OID must not be NULL

- Rule V2: OID must be positive integer ( $> 0$ )

- Rule V3: CID must not be NULL

- Rule V4: CID must be positive integer ( $> 0$ )

- Rule V5: ONAME must not be NULL

- Rule V6: ONAME must not be empty string after trimming whitespace

- Rule V7: OID must be unique (first occurrence retained)

- Error Handling and Logging:

- Validation Failure: Log WARN for each validation rule violation with count

- Error Log Write Failure: Log ERROR "Failed to write error log to S3: {error}" → Continue job execution

- Zero Valid Records: Log WARN "No valid order records after validation" → Continue with empty DataFrame

- High Rejection Rate: If  $\text{invalid\_records} / \text{total\_records} > 0.5$ , log WARN "High rejection rate: {percentage}%"

- Runtime Parameters:

- `validation.orders.error\_log\_path`: s3://adif-sdlc/logs/errors/orders/ (from YAML config)
- `validation.orders.duplicate\_key`: OID (from YAML config)
- Operational Notes:
  - Validation should complete within 5 minutes for 50M records
  - Error log partitioning by run\_date enables efficient historical analysis
  - Consider sampling error log if invalid\_records > 100,000 (write first 100,000 + summary)
  - Drop validation flag columns before proceeding to next stage
- 

### ### TR-TRANSFORM-001: Create Full Customer Name

- Related Functional Requirement ID(s): FR-TRANSFORM-001
- Objective:

Derive FULL\_NAME column by concatenating FNAME and LNAME with proper case (title case) formatting.

- Source Details:
  - Source Type: In-memory Spark DataFrame
  - Source Name: df\_customers\_valid (from TR-VALIDATE-001)
- Target Details:
  - Target Type: In-memory Spark DataFrame
  - Target Name: df\_customers\_transformed
- Input Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL |  |
|  | FNAME       | String    | Variable               | NOT NULL |  |
|  | LNAME       | String    | Variable               | NOT NULL |  |

- Output Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Derivation            |  |
|--|-------------|-----------|------------------------|----------|-----------------------|--|
|  | -----       | -----     | -----                  | -----    | -----                 |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL | Direct map from input |  |
|  | FNAME       | String    | Variable               | NOT NULL | Direct map from input |  |
|  | LNAME       | String    | Variable               | NOT NULL | Direct map from input |  |

|  |           |        |          |          |                                                |  |
|--|-----------|--------|----------|----------|------------------------------------------------|--|
|  | FULL_NAME | String | Variable | NOT NULL | Derived: initcap(FNAME) + ' ' + initcap(LNAME) |  |
|--|-----------|--------|----------|----------|------------------------------------------------|--|

- Processing / Transformation Logic:

1. Trim Whitespace:

...

```
fname_trimmed = trim(col("FNAME"))
```

```
lname_trimmed = trim(col("LNAME"))
```

...

2. Apply Proper Case (Title Case):

...

Use initcap() function (PySpark) or initCap() (Scala):

```
fname_proper = initcap(fname_trimmed)
```

```
lname_proper = initcap(lname_trimmed)
```

...

3. Concatenate with Space:

...

```
full_name = concat(fname_proper, lit(" "), lname_proper)
```

...

4. Add FULL\_NAME Column:

...

```
df_customers_transformed = df_customers_valid.withColumn("FULL_NAME", full_name)
```

...

5. Verify Transformation:

...

Validate FULL\_NAME is not NULL for all records

Log sample records (first 5) for verification

...

- Validation Rules:

- FULL\_NAME must not be NULL
- FULL\_NAME must contain exactly one space character
- FULL\_NAME length must be >= 3 characters (minimum: "A B")
- FULL\_NAME must follow pattern: "Firstname Lastname"

- Error Handling and Logging:

- Null FULL\_NAME: Log ERROR "FULL\_NAME derivation resulted in NULL for CID: {cid}" → Terminate job with exit code 5
- Transformation Failure: Log ERROR "Failed to apply proper case transformation: {error}" → Terminate job with exit code 5
- Success: Log INFO "Successfully derived FULL\_NAME for {count} customer records"
- Runtime Parameters:
  - `transformation.name\_case`: initcap (from YAML config, supports: initcap, upper, lower)
- Operational Notes:
  - initcap() function handles Unicode characters correctly
  - Transformation should complete within 1 minute for 10M records
  - Consider caching df\_customers\_transformed if used multiple times downstream
  - Preserve original FNAME and LNAME columns for audit purposes
- 

### ### TR-TRANSFORM-002: Join Customer and Order Data

- Related Functional Requirement ID(s): FR-TRANSFORM-002
- Objective:

Perform inner join between validated customer and order DataFrames on CID to create integrated customer-order dataset.

- Source Details:
  - Source Type 1: In-memory Spark DataFrame
  - Source Name 1: df\_customers\_transformed (from TR-TRANSFORM-001)
  - Source Type 2: In-memory Spark DataFrame
  - Source Name 2: df\_orders\_valid (from TR-VALIDATE-002)
- Target Details:
  - Target Type: In-memory Spark DataFrame
  - Target Name: df\_customer\_orders\_joined
- Input Schema 1 (Customers):

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL |  |
|  | FNAME       | String    | Variable               | NOT NULL |  |
|  | LNAME       | String    | Variable               | NOT NULL |  |

|  |           |        |          |          |  |
|--|-----------|--------|----------|----------|--|
|  | FULL_NAME | String | Variable | NOT NULL |  |
|--|-----------|--------|----------|----------|--|

• Input Schema 2 (Orders):

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | OID         | Integer   | 32-bit                 | NOT NULL |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL |  |
|  | ONAME       | String    | Variable               | NOT NULL |  |

• Output Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Source              |  |
|--|-------------|-----------|------------------------|----------|---------------------|--|
|  | -----       | -----     | -----                  | -----    | -----               |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL | customers.CID       |  |
|  | FULL_NAME   | String    | Variable               | NOT NULL | customers.FULL_NAME |  |
|  | OID         | Integer   | 32-bit                 | NOT NULL | orders.OID          |  |
|  | ONAME       | String    | Variable               | NOT NULL | orders.ONAME        |  |

• Processing / Transformation Logic:

1. Perform Inner Join:

...

```
df_customer_orders_joined = df_customers_transformed.join(
df_orders_valid,
df_customers_transformed.CID == df_orders_valid.CID,
"inner"
)
...
```

2. Select Required Columns:

...

```
df_customer_orders_joined = df_customer_orders_joined.select(
df_customers_transformed.CID.alias("CID"),
df_customers_transformed.FULL_NAME.alias("FULL_NAME"),
df_orders_valid.OID.alias("OID"),
df_orders_valid.ONAME.alias("ONAME")
)
...
```



### 3. Calculate Join Metrics:

...

```
customer_count = df_customers_transformed.count()
order_count = df_orders_valid.count()
joined_count = df_customer_orders_joined.count()
customers_with_orders = df_customer_orders_joined.select("CID").distinct().count()
customers_without_orders = customer_count - customers_with_orders
orders_without_customers = 0 (by definition of inner join)
```

...

### 4. Log Join Statistics:

...

```
INFO: "Join Statistics:"
INFO: " Total Customers: {customer_count}"
INFO: " Total Orders: {order_count}"
INFO: " Joined Records: {joined_count}"
INFO: " Customers with Orders: {customers_with_orders}"
INFO: " Customers without Orders (excluded): {customers_without_orders}"
```

...

### 5. Verify Referential Integrity:

...

```
Validate: All OID values in joined DataFrame exist in df_orders_valid
Validate: All CID values in joined DataFrame exist in df_customers_transformed
```

...

- Validation Rules:
  - Join must produce at least 1 record (unless both inputs are empty)
  - All CID values in output must exist in customer DataFrame
  - All OID values in output must exist in order DataFrame
  - No NULL values in any output column
- Error Handling and Logging:
  - Empty Join Result: Log WARN "Inner join produced zero records" → Continue with empty DataFrame
  - Referential Integrity Violation: Log ERROR "Referential integrity check failed" → Terminate job with exit code 6
  - Join Failure: Log ERROR "Join operation failed: {error}" → Terminate job with exit code 6

- Success: Log INFO "Successfully joined {joined\_count} customer-order records"
- Runtime Parameters:
  - `transformation.join\_type`: inner (from YAML config, supports: inner, left, right, full)
  - `transformation.join\_key`: CID (from YAML config)
- Operational Notes:
  - Inner join excludes customers without orders (expected behavior per FRD)
  - Join should complete within 5 minutes for 10M customers × 50M orders
  - Consider broadcast join if customer DataFrame < 10MB (use broadcast() hint)
  - Repartition joined DataFrame if skew detected on CID
  - Cache joined DataFrame if used multiple times downstream
- 

### ### TR-TRANSFORM-003: Calculate Order Count per Customer

- Related Functional Requirement ID(s): FR-TRANSFORM-003
- Objective:

Calculate ORDER\_COUNT metric representing total number of orders per customer using window function partitioned by CID.

- Source Details:
  - Source Type: In-memory Spark DataFrame
  - Source Name: df\_customer\_orders\_joined (from TR-TRANSFORM-002)
- Target Details:
  - Target Type: In-memory Spark DataFrame
  - Target Name: df\_customer\_orders\_with\_count
- Input Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL |  |
|  | FULL_NAME   | String    | Variable               | NOT NULL |  |
|  | OID         | Integer   | 32-bit                 | NOT NULL |  |
|  | ONAME       | String    | Variable               | NOT NULL |  |

- Output Schema:

| Column Name | Data Type | Length/Precision/Scale | Nullable | Derivation |  |
|-------------|-----------|------------------------|----------|------------|--|
|-------------|-----------|------------------------|----------|------------|--|

|  |             |         |          |          |                                             |  |
|--|-------------|---------|----------|----------|---------------------------------------------|--|
|  | -----       | -----   | -----    | -----    | -----                                       |  |
|  | CID         | Integer | 32-bit   | NOT NULL | Direct map from input                       |  |
|  | FULL_NAME   | String  | Variable | NOT NULL | Direct map from input                       |  |
|  | OID         | Integer | 32-bit   | NOT NULL | Direct map from input                       |  |
|  | ONAME       | String  | Variable | NOT NULL | Direct map from input                       |  |
|  | ORDER_COUNT | Integer | 32-bit   | NOT NULL | Derived: COUNT(OID) OVER (PARTITION BY CID) |  |

• Processing / Transformation Logic:

1. Define Window Specification:

---

from pyspark.sql.window import Window

window\_spec = Window.partitionBy("CID")

---

2. Calculate Order Count:

---

df\_customer\_orders\_with\_count = df\_customer\_orders\_joined.withColumn(  
"ORDER\_COUNT",  
count("OID").over(window\_spec)  
)

---

3. Verify Calculation:

---

Validate: ORDER\_COUNT >= 1 for all records

Validate: All records with same CID have identical ORDER\_COUNT value

---

4. Calculate Aggregation Metrics:

---

min\_order\_count = df\_customer\_orders\_with\_count.agg(min("ORDER\_COUNT")).collect()[0][0]  
max\_order\_count = df\_customer\_orders\_with\_count.agg(max("ORDER\_COUNT")).collect()[0][0]  
avg\_order\_count = df\_customer\_orders\_with\_count.agg(avg("ORDER\_COUNT")).collect()[0][0]

---

5. Log Calculation Summary:

---

INFO: "Order Count Calculation Summary:"  
INFO: " Min Orders per Customer: {min\_order\_count}"  
INFO: " Max Orders per Customer: {max\_order\_count}"  
INFO: " Avg Orders per Customer: {avg\_order\_count:.2f}"  
...

- Validation Rules:
  - ORDER\_COUNT must be  $\geq 1$  (inner join ensures at least one order per customer)
  - ORDER\_COUNT must be consistent for all records with same CID
  - ORDER\_COUNT must be integer data type
  - ORDER\_COUNT must not be NULL
- Error Handling and Logging:
  - Null ORDER\_COUNT: Log ERROR "ORDER\_COUNT calculation resulted in NULL for CID: {cid}" → Terminate job with exit code 7
  - Invalid ORDER\_COUNT: Log ERROR "ORDER\_COUNT < 1 detected for CID: {cid}" → Terminate job with exit code 7
  - Calculation Failure: Log ERROR "Window function calculation failed: {error}" → Terminate job with exit code 7
  - Success: Log INFO "Successfully calculated ORDER\_COUNT for {customer\_count} customers"
- Runtime Parameters:
  - ``transformation.window_partition_key``: CID (from YAML config)
- Operational Notes:
  - Window function may trigger shuffle operation; monitor Spark UI for performance
  - Calculation should complete within 3 minutes for 50M records
  - Consider repartitioning by CID before window function if data is skewed
  - ORDER\_COUNT includes all orders in joined DataFrame (may include duplicates if source had duplicates before deduplication)
- 

#### ### TR-TRANSFORM-004: Add Processing Timestamp

- Related Functional Requirement ID(s): FR-TRANSFORM-004
- Objective:

Add PROCESSED\_DATE column containing current timestamp at job execution time for audit trail and data lineage tracking.

- Source Details:

- Source Type: In-memory Spark DataFrame
- Source Name: df\_customer\_orders\_with\_count (from TR-TRANSFORM-003)
- Target Details:
  - Target Type: In-memory Spark DataFrame
  - Target Name: df\_customer\_orders\_final
- Input Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CID         | Integer   | 32-bit                 | NOT NULL |  |
|  | FULL_NAME   | String    | Variable               | NOT NULL |  |
|  | OID         | Integer   | 32-bit                 | NOT NULL |  |
|  | ONAME       | String    | Variable               | NOT NULL |  |
|  | ORDER_COUNT | Integer   | 32-bit                 | NOT NULL |  |

- Output Schema:

|  | Column Name    | Data Type | Length/Precision/Scale | Nullable | Derivation                   |  |
|--|----------------|-----------|------------------------|----------|------------------------------|--|
|  | -----          | -----     | -----                  | -----    | -----                        |  |
|  | CID            | Integer   | 32-bit                 | NOT NULL | Direct map from input        |  |
|  | FULL_NAME      | String    | Variable               | NOT NULL | Direct map from input        |  |
|  | OID            | Integer   | 32-bit                 | NOT NULL | Direct map from input        |  |
|  | ONAME          | String    | Variable               | NOT NULL | Direct map from input        |  |
|  | ORDER_COUNT    | Integer   | 32-bit                 | NOT NULL | Direct map from input        |  |
|  | PROCESSED_DATE | Timestamp | Timestamp              | NOT NULL | Derived: current_timestamp() |  |

- Processing / Transformation Logic:

1. Capture Current Timestamp:

```
'''
```

```
from pyspark.sql.functions import current_timestamp
```

```
processing_timestamp = current_timestamp()
```

```
'''
```

2. Add PROCESSED\_DATE Column:

```
'''
```

```
df_customer_orders_final = df_customer_orders_with_count.withColumn(
 "PROCESSED_DATE",
```

```
processing_timestamp
```

```
)
```

```
...
```

### 3. Verify Timestamp:

```
...
```

Validate: PROCESSED\_DATE is not NULL for all records

Validate: All records have identical PROCESSED\_DATE value

Validate: PROCESSED\_DATE format is YYYY-MM-DD HH:MM:SS

```
...
```

### 4. Log Timestamp:

```
...
```

```
timestamp_value = df_customer_orders_final.select("PROCESSED_DATE").first()[0]
```

```
INFO: "Added PROCESSED_DATE: {timestamp_value}"
```

```
...
```

- Validation Rules:

- PROCESSED\_DATE must not be NULL
- PROCESSED\_DATE must be identical for all records in DataFrame
- PROCESSED\_DATE must be in Timestamp data type
- PROCESSED\_DATE must represent actual job execution time

- Error Handling and Logging:

- Null PROCESSED\_DATE: Log ERROR "PROCESSED\_DATE is NULL" → Terminate job with exit code 8
- Timestamp Generation Failure: Log ERROR "Failed to generate timestamp: {error}" → Terminate job with exit code 8
- Success: Log INFO "Successfully added PROCESSED\_DATE to {count} records"

- Runtime Parameters:

- `transformation.timestamp\_column`: PROCESSED\_DATE (from YAML config)
- `transformation.timestamp\_format`: yyyy-MM-dd HH:mm:ss (from YAML config)

- Operational Notes:

- current\_timestamp() uses Spark driver system time
- Timestamp is captured once and applied to all records for consistency
- Transformation should complete within 30 seconds for 50M records
- PROCESSED\_DATE will be used for partitioning in output (TR-LOAD-001)

- 

### TR-LOAD-001: Write Output to S3 in Parquet Format

- Related Functional Requirement ID(s): FR-LOAD-001
- Objective:

Write final transformed dataset to S3 in Parquet format with overwrite mode and partitioning by PROCESSED\_DATE.

- Source Details:
  - Source Type: In-memory Spark DataFrame
  - Source Name: df\_customer\_orders\_final (from TR-TRANSFORM-004)
- Target Details:
  - Target Type: S3 Parquet Files
  - Target Path: s3://adif-sdlc/output/customer\_orders/
  - File Format: Parquet
  - Compression: SNAPPY
  - Write Mode: Overwrite
  - Partitioning: By PROCESSED\_DATE
- Input Schema:

|  | Column Name    | Data Type | Length/Precision/Scale | Nullable |  |
|--|----------------|-----------|------------------------|----------|--|
|  | -----          | -----     | -----                  | -----    |  |
|  | CID            | Integer   | 32-bit                 | NOT NULL |  |
|  | FULL_NAME      | String    | Variable               | NOT NULL |  |
|  | OID            | Integer   | 32-bit                 | NOT NULL |  |
|  | ONAME          | String    | Variable               | NOT NULL |  |
|  | ORDER_COUNT    | Integer   | 32-bit                 | NOT NULL |  |
|  | PROCESSED_DATE | Timestamp | Timestamp              | NOT NULL |  |

- Output Schema:

Same as Input Schema (all columns written to Parquet)

- Processing / Transformation Logic:

1. Validate Pre-Write Conditions:

```

```
input_record_count = df_customer_orders_final.count()
```

```
Validate: input_record_count > 0
Validate: All required columns exist
Validate: No NULL values in any column
...
```

2. Configure Write Options:

```
...

write_options = {
    "mode": "overwrite",
    "format": "parquet",
    "compression": "snappy",
    "partitionBy": "PROCESSED_DATE"
}
...
```

3. Write to S3:

```
...

df_customer_orders_final.write \
    .mode("overwrite") \
    .format("parquet") \
    .option("compression", "snappy") \
    .partitionBy("PROCESSED_DATE") \
    .save("s3://adif-sdlc/output/customer_orders/")
...
```

4. Verify Write Success:

```
...

Read back written data:
df_verification = spark.read.parquet("s3://adif-sdlc/output/customer_orders/")
output_record_count = df_verification.count()

Validate: output_record_count == input_record_count
...
```

5. Calculate Write Metrics:

```
...

partition_count = df_verification.select("PROCESSED_DATE").distinct().count()
file_count = count files in S3 path (via S3 API or Spark)
```


...

6. Log Write Summary:

...

```
INFO: "Write Operation Summary:"
INFO: " Target Path: s3://adif-sdlc/output/customer_orders/"
INFO: " Input Record Count: {input_record_count}"
INFO: " Output Record Count: {output_record_count}"
INFO: " Partition Count: {partition_count}"
INFO: " File Count: {file_count}"
INFO: " Write Mode: overwrite"
INFO: " Format: parquet"
INFO: " Compression: snappy"
```

...

- Validation Rules:
 - Input record count must equal output record count (no data loss)
 - All columns must be present in output
 - Parquet files must be readable and valid
 - Partition structure must follow: processed_date=YYYY-MM-DD HH:MM:SS/
- Error Handling and Logging:
 - Write Failure: Log ERROR "Failed to write data to S3: {error}" → Terminate job with exit code 9
 - Record Count Mismatch: Log ERROR "Data loss detected. Input: {input_count}, Output: {output_count}" → Terminate job with exit code 9
 - S3 Access Denied: Log ERROR "Access denied to S3 path: {path}" → Terminate job with exit code 9
 - Disk Space/Quota Exceeded: Log ERROR "S3 write failed due to quota/space: {error}" → Terminate job with exit code 9
 - Verification Failure: Log ERROR "Output verification failed: {error}" → Terminate job with exit code 9
 - Success: Log INFO "Successfully wrote {output_record_count} records to S3"
- Runtime Parameters:
 - `target.output.path`: s3://adif-sdlc/output/customer_orders/ (from YAML config)
 - `target.output.format`: parquet (from YAML config)
 - `target.output.compression`: snappy (from YAML config)
 - `target.output.mode`: overwrite (from YAML config)

- `target.output.partition\_by`: PROCESSED_DATE (from YAML config)
- Operational Notes:
- Overwrite mode replaces all existing data at target path
- Partitioning by PROCESSED_DATE creates one partition per job run
- SNAPPY compression provides good balance of compression ratio and speed
- Write operation should complete within 10 minutes for 50M records
- Consider coalesce() before write if generating too many small files
- Recommended file size: 128MB - 1GB per file
- Monitor S3 PUT request metrics for throttling
-

TR-ERROR-001: Error Handling and Logging Framework

- Related Functional Requirement ID(s): FR-ERROR-001
- Objective:

Implement comprehensive error handling, rejected record logging, and error reporting framework across all ETL stages.

- Source Details:
- Source Type: Multiple (invalid records from validation stages, job execution events)
- Source Names:
- df_customers_invalid (from TR-VALIDATE-001)
- df_orders_invalid (from TR-VALIDATE-002)
- Job execution logs
- Target Details:
- Error Log Path: s3://adif-sdlc/logs/errors/
- Error Report Path: s3://adif-sdlc/logs/reports/
- Job Execution Log: CloudWatch Logs
- Input Schema (Error Records):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Notes	
	-----	-----	-----	-----	-----	
	[Original Columns]	[Original Types]	[Original Specs]	Variable	All original columns from source	
	rejection_reason	String	Variable	NOT NULL	Validation failure reason	
	error_timestamp	Timestamp	Timestamp	NOT NULL	When error was detected	
	job_run_id	String	Variable	NOT NULL	Unique job execution identifier	

- Output Schema (Error Log):

Same as Input Schema

- Processing / Transformation Logic:

1. Initialize Error Logging Framework:

...

job_run_id = generate_unique_id() (e.g., UUID or timestamp-based)

error_timestamp = current_timestamp()

Configure logging:

- Log level from YAML config (INFO/DEBUG/ERROR)
- Log destination: CloudWatch Logs
- Log format: JSON structured logging

...

2. Capture Validation Errors:

...

For each validation stage (TR-VALIDATE-001, TR-VALIDATE-002):

Add error metadata columns:

- error_timestamp
- job_run_id

Write invalid records to error log:

Path: s3://adif-sdlc/logs/errors/{entity}/run_date={YYYY-MM-DD}/

Format: CSV with header

Partition by: run_date

...

3. Log Job Execution Events:

...

Log at each stage:

- Stage start: INFO "Starting stage: {stage_name}"
- Stage end: INFO "Completed stage: {stage_name}, Duration: {duration}s"
- Record counts: INFO "Processed {count} records"
- Warnings: WARN "{warning_message}"
- Errors: ERROR "{error_message}"

...

4. Generate Error Summary Report:

...

Aggregate error statistics:

- Total records processed by entity
- Total records rejected by entity
- Breakdown by rejection reason
- Top 10 most common errors
- Sample of rejected records (first 100)

Write error report:

Path: s3://adif-sdlc/logs/reports/error_summary_{job_run_id}.json

Format: JSON

...

5. Implement Exception Handling:

...

try:

[ETL stage execution]

except Exception as e:

Log ERROR with full stack trace

Write exception details to error log

Determine if error is recoverable:

- Recoverable: Log WARN and continue
- Non-recoverable: Log ERROR and terminate with appropriate exit code

...

6. Send Job Completion Notification:

...

Prepare notification payload:

- Job status: SUCCESS/FAILURE
- Job run ID
- Start time, end time, duration
- Record counts (input, output, rejected)
- Error summary (if any)
- S3 output path

Send notification via:

- CloudWatch Events (for automation)
- SNS topic (for email/SMS alerts) - if configured

...

- Validation Rules:
 - All error records must have rejection_reason populated
 - All error records must have error_timestamp populated
 - All error records must have job_run_id populated
 - Error log write must not fail the job (best effort)
- Error Handling and Logging:
 - Error Log Write Failure: Log WARN "Failed to write error log: {error}" → Continue job execution
 - Notification Failure: Log WARN "Failed to send notification: {error}" → Continue job execution
 - Logging Framework Failure: Print to stderr and continue (fallback mechanism)
- Runtime Parameters:
 - `logging.level`: INFO (from YAML config, supports: DEBUG, INFO, WARN, ERROR)
 - `logging.error\_log\_path`: s3://adif-sdlc/logs/errors/ (from YAML config)
 - `logging.report\_path`: s3://adif-sdlc/logs/reports/ (from YAML config)
 - `logging.enable\_metrics`: true (from YAML config)
 - `notification.sns\_topic\_arn`: arn:aws:sns:region::glue-job-notifications (from YAML config, optional)
- Operational Notes:
 - Error logs are partitioned by run_date for efficient querying
 - Error reports are stored as JSON for easy parsing and analysis
 - CloudWatch Logs retention: 30 days (configurable)
 - Consider implementing error log sampling if rejection rate > 10% (write first 10,000 + summary)
 - Error handling should not significantly impact job performance (< 5% overhead)
-

TR-CONFIG-001: YAML Configuration Management

- Related Functional Requirement ID(s): FR-CONFIG-001
- Objective:

Implement YAML-based configuration management for all runtime parameters to enable environment-agnostic deployment.

- Source Details:
- Source Type: YAML Configuration File
- Source Path: s3:///config/job_config.yaml
- Target Details:
- Target Type: In-memory Configuration Object
- Target Name: config
- Input Schema (YAML Structure):

```
``yaml
job:
  name: "CustomerOrderETL"
  description: "Customer Order Analytics ETL Pipeline"
  version: "1.0"

  source:
    customers:
      path: "s3://adif-sdlc/busi_req_doc/customers.csv"
      format: "csv"
      schema:
        - name: "CID"
          type: "integer"
          nullable: false
        - name: "FNAME"
          type: "string"
          nullable: false
        - name: "LNAME"
          type: "string"
          nullable: false

    orders:
      path: "s3://adif-sdlc/busi_req_doc/orders.csv"
      format: "csv"
      schema:
        - name: "OID"
          type: "integer"
          nullable: false
```

- name: "CID"

type: "integer"

nullable: false

- name: "ONAME"

type: "string"

nullable: false

target:

output:

path: "s3://adif-sdlc/output/customer_orders/"

format: "parquet"

compression: "snappy"

mode: "overwrite"

partition_by: "PROCESSED_DATE"

validation:

customers:

error_log_path: "s3://adif-sdlc/logs/errors/customers/"

duplicate_key: "CID"

orders:

error_log_path: "s3://adif-sdlc/logs/errors/orders/"

duplicate_key: "OID"

transformation:

name_case: "initcap"

join_type: "inner"

join_key: "CID"

window_partition_key: "CID"

timestamp_column: "PROCESSED_DATE"

timestamp_format: "yyyy-MM-dd HH:mm:ss"

logging:

level: "INFO"

error_log_path: "s3://adif-sdlc/logs/errors/"

report_path: "s3://adif-sdlc/logs/reports/"

enable_metrics: true

```
notification:
sns_topic_arn: "arn:aws:sns:region::glue-job-notifications"
enable: false

spark:
app_name: "CustomerOrderETL"
sql_files_max_partition_bytes: "134217728" # 128MB
sql_adaptive_enabled: "true"
sql_adaptive_coalesce_partitions_enabled: "true"
...

```

- Output Schema:

Configuration object with nested structure matching YAML

- Processing / Transformation Logic:

1. Load YAML Configuration:

```
...
import yaml

config_path = "s3:///config/job_config.yaml"

# Read YAML file from S3
config_content = read_s3_file(config_path)

# Parse YAML
config = yaml.safe_load(config_content)
...

```

2. Validate Configuration Schema:

```
...
```

Required sections:

- job
- source (customers, orders)
- target (output)
- validation
- transformation
- logging

Required parameters:

- source.customers.path
- source.orders.path
- target.output.path
- target.output.format
- target.output.mode

Validate data types:

- Paths must be strings starting with "s3://"
- Formats must be in allowed list: ["csv", "parquet", "json"]
- Mode must be in allowed list: ["overwrite", "append"]
- Logging level must be in allowed list: ["DEBUG", "INFO", "WARN", "ERROR"]

...

3. Apply Configuration to Job:

...

Use config values throughout job execution:

- Source paths: config['source']['customers']['path']
- Target path: config['target']['output']['path']
- Write mode: config['target']['output']['mode']
- Logging level: config['logging']['level']
- etc.

...

4. Log Configuration Summary:

...

INFO: "Job Configuration Loaded:"

INFO: " Job Name: {config['job']['name']}"

INFO: " Job Version: {config['job']['version']}"

INFO: " Source Customers: {config['source']['customers']['path']}"

INFO: " Source Orders: {config['source']['orders']['path']}"

INFO: " Target Output: {config['target']['output']['path']}"

INFO: " Write Mode: {config['target']['output']['mode']}"

INFO: " Logging Level: {config['logging']['level']}"

Do not log sensitive values (credentials, secrets)

...

5. Support Environment-Specific Overrides:

...

Allow command-line arguments to override YAML values:

- config-path: Override default config file path
- source-customers-path: Override source.customers.path
- target-output-path: Override target.output.path
- etc.

Priority: Command-line args > YAML config > Default values

...

- Validation Rules:
 - YAML file must exist at specified path
 - YAML must be valid and parsable
 - All required sections and parameters must be present
 - Parameter values must match expected data types
 - S3 paths must be valid and accessible
- Error Handling and Logging:
 - Config File Not Found: Log ERROR "Configuration file not found: {path}" → Terminate job with exit code 10
 - YAML Parsing Error: Log ERROR "Failed to parse YAML configuration: {error}" → Terminate job with exit code 10
 - Missing Required Parameter: Log ERROR "Required parameter missing: {parameter}" → Terminate job with exit code 10
 - Invalid Parameter Value: Log ERROR "Invalid value for parameter {parameter}: {value}" → Terminate job with exit code 10
 - S3 Access Denied: Log ERROR "Access denied to configuration file: {path}" → Terminate job with exit code 10
 - Success: Log INFO "Configuration loaded successfully from {path}"
- Runtime Parameters:
 - `--config-path`: Path to YAML configuration file (command-line argument)
 - Default: `s3:///config/job_config.yaml`
- Operational Notes:
 - Configuration file should be version-controlled
 - Use separate config files for dev/test/prod environments
 - Validate configuration file syntax before deployment
 - Consider using AWS Secrets Manager for sensitive values (not stored in YAML)

- Configuration loading should complete within 10 seconds
-

TR-TEST-001: Unit Testing Framework

- Related Functional Requirement ID(s): FR-TEST-001
- Objective:

Implement comprehensive unit test suite covering all transformation logic, validation rules, and error handling using pytest (Python) or ScalaTest (Scala).

- Source Details:
- Source Type: Test Data Files
- Source Path: s3://adif-sdlc/test_data/unit/
- Target Details:
- Target Type: Test Results Report
- Target Path: test_results/unit_test_report.xml (JUnit XML format)
- Test Coverage Requirements:

1. Test Data Loading (TR-EXTRACT-001, TR-EXTRACT-002):

...

Test Cases:

- test_load_customers_success: Verify successful CSV loading
- test_load_customers_file_not_found: Verify error handling for missing file
- test_load_customers_empty_file: Verify handling of empty file
- test_load_customers_schema_mismatch: Verify error handling for schema mismatch
- test_load_customers_malformed_csv: Verify error handling for malformed CSV
- test_load_orders_success: Verify successful CSV loading
- test_load_orders_file_not_found: Verify error handling for missing file
- test_load_orders_empty_file: Verify handling of empty file

...

2. Test Validation Logic (TR-VALIDATE-001, TR-VALIDATE-002):

...

Test Cases:

- test_validate_customers_null_cid: Verify rejection of NULL CID
- test_validate_customers_invalid_cid: Verify rejection of CID <= 0

- test_validate_customers_null_fname: Verify rejection of NULL FNAME
- test_validate_customers_empty_fname: Verify rejection of empty FNAME
- test_validate_customers_null_lname: Verify rejection of NULL LNAME
- test_validate_customers_empty_lname: Verify rejection of empty LNAME
- test_validate_customers_duplicates: Verify duplicate removal
- test_validate_customers_all_valid: Verify all valid records pass
- test_validate_orders_null_oid: Verify rejection of NULL OID
- test_validate_orders_invalid_oid: Verify rejection of $OID \leq 0$
- test_validate_orders_null_cid: Verify rejection of NULL CID
- test_validate_orders_invalid_cid: Verify rejection of $CID \leq 0$
- test_validate_orders_null_ename: Verify rejection of NULL ENAME
- test_validate_orders_empty_ename: Verify rejection of empty ENAME
- test_validate_orders_duplicates: Verify duplicate removal

...

3. Test Transformation Logic (TR-TRANSFORM-001, TR-TRANSFORM-003, TR-TRANSFORM-004):

...

Test Cases:

- test_full_name_proper_case: Verify proper case conversion
- test_full_name_lowercase_input: Verify conversion from lowercase
- test_full_name_uppercase_input: Verify conversion from uppercase
- test_full_name_mixed_case_input: Verify conversion from mixed case
- test_full_name_whitespace_trimming: Verify whitespace trimming
- test_order_count_single_order: Verify count = 1 for single order
- test_order_count_multiple_orders: Verify correct count for multiple orders
- test_order_count_consistency: Verify same count for all records of same customer
- test_processed_date_not_null: Verify PROCESSED_DATE is not NULL
- test_processed_date_consistency: Verify same timestamp for all records

...

4. Test Join Logic (TR-TRANSFORM-002):

...

Test Cases:

- test_join_matching_records: Verify successful join with matching records
- test_join_no_matches: Verify empty result when no matches

- test_join_multiple_orders_per_customer: Verify correct join with 1:N relationship
- test_join_referential_integrity: Verify all joined records have valid references
- test_join_customer_without_orders: Verify customer excluded (inner join)
- test_join_order_without_customer: Verify order excluded (inner join)

...

5. Test Error Handling (TR-ERROR-001):

...

Test Cases:

- test_error_logging_validation_failure: Verify error log creation
- test_error_logging_rejection_reason: Verify rejection reason populated
- test_error_handling_continue_on_validation_error: Verify job continues after validation error
- test_error_handling_terminate_on_critical_error: Verify job terminates on critical error

...

- Test Data Specifications:

...

Sample customers.csv (8 records):

CID,FNAME,LNAME

1,John,doe # Valid - test proper case
 2,JANE,SMITH # Valid - test proper case
 3,Bob,Johnson # Valid - already proper case
 4,,Williams # Invalid - empty FNAME
 5,Alice, # Invalid - empty LNAME
 6,Charlie,Brown # Valid
 7,Charlie,Brown # Duplicate - should be removed
 8,-1,Invalid # Invalid - CID <= 0

Sample orders.csv (12 records):

OID,CID,ONAME

101,1,Order A # Valid
 102,1,Order B # Valid - multiple orders for CID 1
 103,2,Order C # Valid
 104,3,Order D # Valid
 105,3,Order E # Valid - multiple orders for CID 3
 106,3,Order F # Valid - multiple orders for CID 3

107,6,Order G # Valid
108,6,Order H # Valid - multiple orders for CID 6
109,,Order I # Invalid - NULL CID
110,99,Order J # Valid but no matching customer (excluded in join)
111,6,Order K # Valid - multiple orders for CID 6
112,6,Order K # Duplicate - should be removed
...

- Test Execution Framework:

```
```python
pytest structure (Python)
tests/
unit/
test_extract.py
test_validate.py
test_transform.py
test_join.py
test_error_handling.py
conftest.py # Shared fixtures

Run tests:
pytest tests/unit/ --junitxml=test_results/unit_test_report.xml --cov=src --cov-report=html
```
```

- Validation Rules:
 - All unit tests must pass (100% pass rate)
 - Code coverage must be $\geq 80\%$
 - Test execution time must be < 5 minutes
 - Tests must be deterministic (no flaky tests)
- Error Handling and Logging:
 - Test Failure: Report failure details with assertion error message
 - Test Coverage Below Threshold: Report warning with coverage percentage
 - Test Execution Timeout: Report error and terminate test run
- Runtime Parameters:
 - `--test-data-path``: Path to test data files (default: s3://adif-sdlc/test_data/unit/)

- `--coverage-threshold`: Minimum code coverage percentage (default: 80)

- Operational Notes:

- Run unit tests before every deployment
- Integrate with CI/CD pipeline for automated testing
- Use mocking for external dependencies (S3, Glue Catalog)
- Maintain test data files in version control
- Update tests when business logic changes

-

TR-TEST-002: Integration Testing Framework

- Related Functional Requirement ID(s): FR-TEST-002

- Objective:

Implement end-to-end integration testing with sample data to validate complete job execution flow and output correctness.

- Source Details:

- Source Type: Test Data Files
- Source Path: s3://adif-sdlc/test_data/integration/

- Target Details:

- Target Type: Test Results Report
- Target Path: test_results/integration_test_report.xml (JUnit XML format)
- Test Output Path: s3://adif-sdlc/test_output/integration/

- Test Scenarios:

1. Test End-to-End Execution:

...

Test Case: test_e2e_happy_path

Description: Verify complete job execution with valid data

Input:

- customers.csv: 10 valid customer records
- orders.csv: 15 valid order records

Expected Output:

- Output files created at s3://adif-sdlc/test_output/integration/
- Output schema matches specification

- Record count = 15 (all orders with matching customers)
- All transformations applied correctly

Validation:

- Verify output file exists
- Verify output schema
- Verify record count
- Verify FULL_NAME format
- Verify ORDER_COUNT accuracy
- Verify PROCESSED_DATE populated

...

2. Test Edge Cases:

...

Test Case: test_e2e_customers_without_orders

Description: Verify customers without orders are excluded

Input:

- customers.csv: 5 customers (CID 1-5)
- orders.csv: 3 orders (CID 1, 2, 3 only)

Expected Output:

- Record count = 3
- Only CID 1, 2, 3 in output
- CID 4, 5 excluded
-

Test Case: test_e2e_duplicate_customers

Description: Verify duplicate customer records are removed

Input:

- customers.csv: 8 customers with 2 duplicates (CID 1, 3)
- orders.csv: 10 orders

Expected Output:

- Unique customers only
- First occurrence of duplicate retained
- All orders joined correctly

-

Test Case: test_e2e_duplicate_orders

Description: Verify duplicate order records are removed

Input:

- customers.csv: 5 customers
- orders.csv: 12 orders with 2 duplicates (OID 101, 105)

Expected Output:

- Unique orders only
- First occurrence of duplicate retained
- ORDER_COUNT reflects unique orders
-

Test Case: test_e2e_null_values

Description: Verify NULL values are rejected and logged

Input:

- customers.csv: 10 customers with 2 NULL CID, 1 NULL FNAME
- orders.csv: 15 orders with 1 NULL OID, 2 NULL CID

Expected Output:

- Invalid records excluded from output
- Error log created with rejection reasons
- Valid records processed successfully
-

Test Case: test_e2e_invalid_data_types

Description: Verify invalid data types are rejected

Input:

- customers.csv: 10 customers with 2 negative CID
- orders.csv: 15 orders with 1 negative OID, 1 negative CID

Expected Output:

- Invalid records excluded from output
- Error log created with rejection reasons
- Valid records processed successfully

...

3. Test Output Schema:

...

Test Case: test_output_schema_validation

Description: Verify output schema matches specification

Validation:

- Column names: CID, FULL_NAME, OID, ONAME, ORDER_COUNT, PROCESSED_DATE
- Column order matches specification
- Data types: CID (Int), FULL_NAME (String), OID (Int), ONAME (String), ORDER_COUNT (Int), PROCESSED_DATE (Timestamp)
- No NULL values in any column

...

4. Test Data Accuracy:

...

Test Case: test_full_name_accuracy

Description: Verify FULL_NAME is correctly derived

Input:

- Customer: CID=1, FNAME="john", LNAME="doe"

Expected Output:

- FULL_NAME = "John Doe"
-

Test Case: test_order_count_accuracy

Description: Verify ORDER_COUNT is accurate

Input:

- Customer: CID=1
- Orders: OID=101,102,103 (all CID=1)

Expected Output:

- All 3 records have ORDER_COUNT = 3
-

Test Case: test_processed_date_accuracy

Description: Verify PROCESSED_DATE is populated

Expected Output:

- PROCESSED_DATE is not NULL
- PROCESSED_DATE is same for all records
- PROCESSED_DATE format: YYYY-MM-DD HH:MM:SS

...

- Test Data Specifications:

...

Integration Test Data Set 1 (Happy Path):

customers.csv (10 records):

CID,FNAME,LNAME

1,John,Doe

2,Jane,Smith

3,Bob,Johnson

4,Alice,Williams

5,Charlie,Brown

6,David,Miller

7,Emma,Davis

8,Frank,Garcia

9,Grace,Martinez

10,Henry,Lopez

orders.csv (15 records):

OID,CID,ONAME

101,1,Order A

102,1,Order B

103,2,Order C

104,3,Order D

105,3,Order E

106,4,Order F

107,5,Order G

108,5,Order H

109,5,Order I

110,6,Order J

111,7,Order K

112,8,Order L

113,9,Order M

114,9,Order N

115,10,Order O

Expected Output: 15 records

Expected ORDER_COUNT distribution:

- CID 1: 2 orders
- CID 2: 1 order
- CID 3: 2 orders
- CID 4: 1 order
- CID 5: 3 orders
- CID 6: 1 order
- CID 7: 1 order
- CID 8: 1 order
- CID 9: 2 orders
- CID 10: 1 order

...

- Test Execution Framework:

```
```python
```

```
pytest structure (Python)
```

```
tests/
```

```
integration/
```

```
test_e2e_happy_path.py
```

```
test_e2e_edge_cases.py
```

```
test_output_validation.py
```

```
test_data_accuracy.py
```

```
conftest.py # Shared fixtures
```

```
Run tests:
```

```
pytest tests/integration/ --junitxml=test_results/integration_test_report.xml
```

```
```
```

- Validation Rules:
- All integration tests must pass (100% pass rate)
- Output schema must match specification exactly

- Output record counts must match expectations
- Data accuracy checks must pass for all test cases
- Edge cases must be handled correctly
- Error Handling and Logging:
 - Test Failure: Report failure details with expected vs actual comparison
 - Output Schema Mismatch: Report schema differences
 - Record Count Mismatch: Report expected vs actual counts
 - Data Accuracy Failure: Report specific records with incorrect values
- Runtime Parameters:
 - `--test-data-path`: Path to integration test data (default: s3://adif-sdlc/test_data/integration/)
 - `--test-output-path`: Path for test output (default: s3://adif-sdlc/test_output/integration/)
 - `--cleanup-after-test`: Clean up test output after test (default: true)
- Operational Notes:
 - Run integration tests before production deployment
 - Use separate S3 paths for test data and output
 - Clean up test output after successful test run
 - Integration tests should complete within 10 minutes
 - Use smaller data volumes for faster test execution
 - Maintain expected output datasets for comparison
 -

3. RUNTIME STRATEGY NOTES

3.1 Authoring Language

- Primary Language: Python 3.7+ with PySpark
- Alternative Language: Scala 2.12 with Spark
- Recommendation: Python for faster development and easier maintenance
- Rationale: Python provides better readability, extensive library support, and easier integration with AWS services

3.2 Execution Strategy

- Execution Mode: AWS Glue ETL Job (Spark)
- Glue Version: 3.0 or higher
- Spark Version: 3.1.1 or higher
- Worker Type: G.1X (recommended) or G.2X for larger datasets

- Number of Workers:
- Development: 2-5 workers
- Production: 5-10 workers (adjust based on data volume)
- Job Timeout: 60 minutes (2880 DPU-hours max)
- Max Concurrent Runs: 1 (prevent overlapping executions)
- Job Bookmark: Disabled (full refresh with overwrite mode)
- Execution Frequency: Daily batch (scheduled via EventBridge/CloudWatch Events)

3.3 Glue Job Type Expectations

- Job Type: Glue ETL (Spark)
- Script Location: s3:///scripts/customer_order_etl.py
- Temporary Directory: s3:///temp/
- Job Parameters:
- `--config-path` : s3:///config/job_config.yaml`
- `--job-bookmark-option` : job-bookmark-disable`
- `--enable-metrics` : true`
- `--enable-continuous-cloudwatch-log` : true`
- `--enable-spark-ui` : true`
- `--spark-event-logs-path` : s3:///spark-logs/`
- IAM Role Requirements:
- s3:GetObject on source paths and config path
- s3:PutObject on target paths, log paths, and temp directory
- glue:GetTable, glue:CreateTable, glue:UpdateTable (if using Glue Catalog)
- logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents (CloudWatch Logs)
- cloudwatch:PutMetricData (CloudWatch Metrics)
-

4. DATA DESIGN DETAILS

4.1 Source Systems and Paths

- Source 1: Customer Master Data
- Source System: S3 CSV File
- Source Path: s3://adif-sdlc/busi_req_doc/customers.csv
- File Format: CSV
- Delimiter: Comma (,)
- Header: Present (first row)

- Encoding: UTF-8
 - Compression: None
 - Expected File Size: < 100MB
 - Expected Record Count: < 10 million
 - Update Frequency: Daily full refresh
 - Data Ownership: Customer Master Data Management Team
-
- Source 2: Order Transaction Data
 - Source System: S3 CSV File
 - Source Path: s3://adif-sdlc/busi_req_doc/orders.csv
 - File Format: CSV
 - Delimiter: Comma (,)
 - Header: Present (first row)
 - Encoding: UTF-8
 - Compression: None
 - Expected File Size: < 500MB
 - Expected Record Count: < 50 million
 - Update Frequency: Daily full refresh
 - Data Ownership: Order Management System Team

4.2 Target Systems and Paths

- Target: Customer Order Analytics Dataset
- Target System: S3 Parquet Files
- Target Path: s3://adif-sdlc/output/customer_orders/
- File Format: Parquet
- Compression: SNAPPY
- Partitioning: By PROCESSED_DATE
- Partition Structure: processed_date=YYYY-MM-DD HH:MM:SS/
- Expected File Size: 128MB - 1GB per file
- Expected Record Count: < 50 million (same as order count after inner join)
- Update Frequency: Daily full refresh (overwrite mode)
- Data Consumers: Analytics Team, Reporting Team, Data Science Team

4.3 Catalog Registration Expectations

- Glue Data Catalog Registration:
- Database Name: customer_analytics

- Table Name: customer_orders
- Table Type: EXTERNAL_TABLE
- Location: s3://adif-sdlc/output/customer_orders/
- SerDe: org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe
- Input Format: org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat
- Output Format: org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat
- Partition Keys: processed_date (timestamp)
- Table Properties:
 - classification: parquet
 - compressionType: snappy
 - typeOfData: file
 - Update Strategy:
 - Create table if not exists
 - Add new partition after each job run
 - Update table statistics
- Catalog Registration Code:

```
```python
```

```
Register table in Glue Catalog
```

```
glueContext.create_dynamic_frame.from_catalog(
 database="customer_analytics",
 table_name="customer_orders",
 transformation_ctx="datasource"
)
```

```
Add partition
```

```
glueContext.add_partition(
 database="customer_analytics",
 table_name="customer_orders",
 partition_input={
 'Values': [processed_date_value],
 'StorageDescriptor': {
 'Location': f's3://adif-sdlc/output/customer_orders/processed_date={processed_date_value}'
 }
 }
)
```



...

#### ### 4.4 Partitioning Strategy

- Partitioning Approach:
- Partition Column: PROCESSED\_DATE (Timestamp)
- Partition Granularity: Per job run (typically daily)
- Partition Format: processed\_date=YYYY-MM-DD HH:MM:SS
- Partition Pruning: Enabled (queries can filter by processed\_date)
- Partition Count: 1 partition per job run (accumulates over time)
- Partition Management:
- Retain partitions for 90 days (configurable)
- Archive older partitions to Glacier (optional)
- Drop partitions older than retention period (optional)
- Partitioning Benefits:
- Efficient querying by processing date
- Easy identification of latest data
- Simplified data lifecycle management
- Reduced query costs (partition pruning)
- Partitioning Considerations:
- Each partition represents a complete snapshot (overwrite mode)
- Historical analysis requires querying specific partition
- Latest data always in most recent partition

#### ### 4.5 File Formats and Table Formats

- Source File Format:
- Format: CSV (Comma-Separated Values)
- Advantages: Human-readable, widely supported, simple structure
- Disadvantages: No schema enforcement, larger file size, slower read performance
- Use Case: Source data from external systems
- Target File Format:
- Format: Parquet (Columnar Storage)
- Compression: SNAPPY
- Advantages:
- Columnar storage for efficient querying

- Built-in compression (SNAPPY provides good balance)
- Schema embedded in file
- Predicate pushdown support
- Efficient for analytical queries
- Disadvantages: Not human-readable, requires Parquet-compatible tools
- Use Case: Optimized for analytics and reporting
- Compression Comparison:

	Codec	Compression Ratio	Speed	CPU Usage	Use Case	
	-----	-----	-----	-----	-----	
	SNAPPY	Medium	Fast	Low	Default choice, good balance	
	GZIP	High	Slow	High	Maximum compression, slower queries	
	UNCOMPRESSED	None	Fastest	None	Testing, small datasets	

- Recommendation: SNAPPY for production (default)

#### ### 4.6 Update/Overwrite/Append Behavior

- Write Mode: OVERWRITE
- Behavior:
  - Each job run completely replaces data at target path
  - Previous data is deleted before new data is written
  - Atomic operation (all-or-nothing)
  - No incremental updates or merges
- Implications:
  - Simple and predictable behavior
  - No need for deduplication or merge logic
  - Historical data not preserved at target location
  - Requires separate archival strategy if history needed
- Alternative Approaches (Not Implemented):
  - APPEND Mode: Add new data without deleting existing data
  - Use case: Incremental loads, event streaming
  - Requires deduplication logic
  - Partition-level append possible
  - MERGE/UPSERT Mode: Update existing records, insert new records
  - Use case: SCD Type 1, incremental updates

- Requires Delta Lake or Iceberg table format
- More complex implementation
- Rationale for OVERWRITE:
  - Aligns with daily full refresh requirement
  - Simplifies job logic (no state management)
  - Ensures data consistency (no partial updates)
  - Matches source data update pattern (full refresh)
- 

## ## 5. PROCESSING DETAILS

### ### 5.1 Data Quality Rules

- Customer Data Quality Rules:

	Rule ID	Rule Description	Validation Logic	Action on Failure	
	-----	-----	-----	-----	
	DQ-C-001	CID must not be NULL	`CID IS NULL`	Reject record, log error	
	DQ-C-002	CID must be positive integer	`CID <= 0`	Reject record, log error	
	DQ-C-003	FNAME must not be NULL	`FNAME IS NULL`	Reject record, log error	
	DQ-C-004	FNAME must not be empty	`trim(FNAME) = ```	Reject record, log error	
	DQ-C-005	LNAME must not be NULL	`LNAME IS NULL`	Reject record, log error	
	DQ-C-006	LNAME must not be empty	`trim(LNAME) = ```	Reject record, log error	

- Order Data Quality Rules:

	Rule ID	Rule Description	Validation Logic	Action on Failure	
	-----	-----	-----	-----	
	DQ-O-001	OID must not be NULL	`OID IS NULL`	Reject record, log error	
	DQ-O-002	OID must be positive integer	`OID <= 0`	Reject record, log error	
	DQ-O-003	CID must not be NULL	`CID IS NULL`	Reject record, log error	
	DQ-O-004	CID must be positive integer	`CID <= 0`	Reject record, log error	
	DQ-O-005	ONAME must not be NULL	`ONAME IS NULL`	Reject record, log error	
	DQ-O-006	ONAME must not be empty	`trim(ONAME) = ```	Reject record, log error	

- Data Quality Metrics:
  - Total records processed
  - Valid records count
  - Invalid records count
  - Rejection rate (invalid / total)

- Breakdown by rejection reason
- Data Quality Thresholds:
  - Warning threshold: Rejection rate > 10%
  - Critical threshold: Rejection rate > 50%
  - Action: Log warning/error, send notification if threshold exceeded

### ### 5.2 Deduplication Rules

- Customer Deduplication:
  - Deduplication Key: CID
  - Strategy: Keep first occurrence, remove subsequent duplicates
  - Implementation: ``dropDuplicates(["CID"])`
  - Ordering: No specific ordering (first occurrence in DataFrame)
  - Logging: Log count of duplicates removed
- Order Deduplication:
  - Deduplication Key: OID
  - Strategy: Keep first occurrence, remove subsequent duplicates
  - Implementation: ``dropDuplicates(["OID"])`
  - Ordering: No specific ordering (first occurrence in DataFrame)
  - Logging: Log count of duplicates removed
- Deduplication Timing:
  - Applied after validation, before transformation
  - Ensures unique keys for join operation
  - Prevents duplicate records in output
- Deduplication Metrics:
  - Total records before deduplication
  - Duplicate records removed
  - Unique records after deduplication
  - Duplication rate (duplicates / total)

### ### 5.3 Join Rules

- Join Type: INNER JOIN
- Join Key: CID (Customer ID)
- Join Logic:

```
```sql
SELECT
c.CID,
c.FULL_NAME,
o.OID,
o.ONAME
FROM customers c
INNER JOIN orders o
ON c.CID = o.CID
```
```

- Join Behavior:
  - Only customers with at least one order are included
  - Customers without orders are excluded
  - Orders without matching customer are excluded
  - One-to-many relationship: One customer can have multiple orders
  - Result: One row per order with customer information
- Join Optimization:
  - Consider broadcast join if customer DataFrame < 10MB
  - Use broadcast hint: `broadcast(df\_customers)`
  - Monitor join skew (uneven distribution of CID values)
  - Repartition by CID if skew detected
- Join Validation:
  - Verify all CID values in output exist in customer DataFrame
  - Verify all OID values in output exist in order DataFrame
  - Verify no NULL values in join key (CID)
  - Log join statistics (customer count, order count, joined count)

### ### 5.4 Aggregation Rules

- Aggregation: Order Count per Customer
- Aggregation Logic:

```
```sql
SELECT
CID,
```

```

FULL_NAME,
OID,
ONAME,
COUNT(OID) OVER (PARTITION BY CID) AS ORDER_COUNT
FROM joined_data
```

```

- Aggregation Method: Window function (not GROUP BY)
- Rationale:
  - Preserve detail-level records (one row per order)
  - Add aggregated metric (ORDER\_COUNT) to each row
  - Avoid collapsing data to customer level
- Aggregation Validation:
  - ORDER\_COUNT >= 1 for all records (inner join ensures at least one order)
  - ORDER\_COUNT is consistent for all records with same CID
  - ORDER\_COUNT matches actual count of orders per customer
- Alternative Approach (Not Implemented):
  - GROUP BY CID to create customer-level summary
  - Would lose order detail (OID, ONAME)
  - Not aligned with FRD requirement (preserve order detail)

### ### 5.5 SCD / History Tracking Rules

- SCD Implementation: Not applicable
- Rationale:
  - FRD specifies overwrite mode (full refresh)
  - No requirement for historical tracking
  - No requirement for change data capture
  - Simplifies implementation and maintenance
- Future Consideration:
  - If historical tracking needed, implement SCD Type 2:
    - Add effective\_date, end\_date, is\_current columns
    - Append new records instead of overwrite
    - Update end\_date for changed records
    - Requires Delta Lake or Iceberg table format

### ### 5.6 Null Handling

- Null Handling Strategy: Reject records with NULL values in required fields
- Required Fields (NOT NULL):
  - Customer: CID, FNAME, LNAME
  - Order: OID, CID, ONAME
- Null Handling Rules:
  - NULL in required field → Reject record, log error
  - NULL in optional field → Not applicable (no optional fields)
  - Empty string treated as invalid (equivalent to NULL for string fields)
- Null Handling Implementation:

```
```python
```

```
# Check for NULL values
```

```
is_null = col("FIELD_NAME").isNull()
```

```
# Check for empty strings
```

```
is_empty = (col("FIELD_NAME").isNotNull()) & (trim(col("FIELD_NAME")) == "")
```

```
# Combined validation
```

is_invalid = is_null	is_empty
----------------------	----------

```
```
```

- Null Handling in Transformations:
  - FULL\_NAME: Cannot be NULL (derived from non-NULL FNAME and LNAME)
  - ORDER\_COUNT: Cannot be NULL (window function on non-NULL OID)
  - PROCESSED\_DATE: Cannot be NULL (generated timestamp)
- Null Handling in Output:
  - Validate no NULL values in any output column before write
  - Terminate job if NULL values detected in output

### ### 5.7 Type Casting Expectations

- Type Casting Requirements:
  - Source to DataFrame:
    - CID: String (CSV) → Integer (DataFrame)
    - OID: String (CSV) → Integer (DataFrame)

- FNAME: String (CSV) → String (DataFrame) [no casting]
- LNAME: String (CSV) → String (DataFrame) [no casting]
- ONAME: String (CSV) → String (DataFrame) [no casting]

- Type Casting Implementation:

```
```python
# Explicit schema definition
schema = StructType([
    StructField("CID", IntegerType(), False),
    StructField("FNAME", StringType(), False),
    StructField("LNAME", StringType(), False)
])

# Read with schema enforcement
df = spark.read.csv(path, schema=schema, header=True)
```
```

- Type Casting Validation:
  - Verify successful casting (no NULL values after cast)
  - Reject records that fail type casting
  - Log casting errors with original value
- Type Casting Errors:
  - Non-numeric value in integer field → Reject record, log error
  - Value out of integer range → Reject record, log error
- Derived Field Types:
  - FULL\_NAME: String (concatenation of strings)
  - ORDER\_COUNT: Integer (count aggregation)
  - PROCESSED\_DATE: Timestamp (current\_timestamp function)
- Output Type Validation:
  - Verify all output columns have expected data types
  - Verify no implicit type conversions during write
  - Parquet format preserves data types
- 

## ## 6. TRACEABILITY MATRIX



### ### 6.1 FRD to TRD Requirement Mapping

| FRD Requirement ID | FRD Requirement Title                | TRD Requirement ID(s) | TRD Requirement Title(s)             |
|--------------------|--------------------------------------|-----------------------|--------------------------------------|
| -----              | -----                                | -----                 | -----                                |
| FR-EXTRACT-001     | Extract Customer Data from S3        | TR-EXTRACT-001        | Extract Customer Data from S3        |
| FR-EXTRACT-002     | Extract Order Data from S3           | TR-EXTRACT-002        | Extract Order Data from S3           |
| FR-VALIDATE-001    | Validate Customer Data Quality       | TR-VALIDATE-001       | Validate Customer Data Quality       |
| FR-VALIDATE-002    | Validate Order Data Quality          | TR-VALIDATE-002       | Validate Order Data Quality          |
| FR-TRANSFORM-001   | Create Full Customer Name            | TR-TRANSFORM-001      | Create Full Customer Name            |
| FR-TRANSFORM-002   | Join Customer and Order Data         | TR-TRANSFORM-002      | Join Customer and Order Data         |
| FR-TRANSFORM-003   | Calculate Order Count per Customer   | TR-TRANSFORM-003      | Calculate Order Count per Customer   |
| FR-TRANSFORM-004   | Add Processing Timestamp             | TR-TRANSFORM-004      | Add Processing Timestamp             |
| FR-LOAD-001        | Write Output to S3 in Parquet Format | TR-LOAD-001           | Write Output to S3 in Parquet Format |
| FR-ERROR-001       | Error Handling and Logging           | TR-ERROR-001          | Error Handling and Logging Framework |
| FR-CONFIG-001      | YAML-Based Configuration Management  | TR-CONFIG-001         | YAML Configuration Management        |
| FR-TEST-001        | Unit Testing Requirements            | TR-TEST-001           | Unit Testing Framework               |
| FR-TEST-002        | Integration Testing Requirements     | TR-TEST-002           | Integration Testing Framework        |

### ### 6.2 Source to Technical Source Mapping

|                     | Technical Source (TRD)                                                             | Source Type      | Notes                  |
|---------------------|------------------------------------------------------------------------------------|------------------|------------------------|
|                     | -----                                                                              | -----            | -----                  |
| customers.csv       | s3://adif-sdlc/busi_req_doc/customers.csv                                          | S3 CSV File      | Direct Source          |
| orders.csv          | s3://adif-sdlc/busi_req_doc/orders.csv                                             | S3 CSV File      | Direct Source          |
| (CID, FNAME, LNAME) | df_customers_raw (CID, FNAME, LNAME)                                               | Spark DataFrame  | Intermediate           |
| (OID, CID, ONAME)   | df_orders_raw (OID, CID, ONAME)                                                    | Spark DataFrame  | Intermediate           |
|                     | df_customers_valid (CID, FNAME, LNAME)                                             | Spark DataFrame  | After Validation       |
|                     | df_orders_valid (OID, CID, ONAME)                                                  | Spark DataFrame  | After Validation       |
|                     | df_customers_transformed (CID, FNAME, LNAME, FULL_NAME)                            | Spark DataFrame  | After Transformation   |
|                     | df_customer_orders_joined (CID, FULL_NAME, OID, ONAME)                             | Spark DataFrame  | After Join             |
|                     | df_customer_orders_with_count (CID, FULL_NAME, OID, ONAME, ORDER_COUNT)            | Spark DataFrame  | After Aggregation      |
|                     | df_customer_orders_final (CID, FULL_NAME, OID, ONAME, ORDER_COUNT, PROCESSED_DATE) | Spark DataFrame  | After Final Processing |
|                     | s3://adif-sdlc/output/customer_orders/                                             | S3 Parquet Files | Final Output           |

- Critical Note: All source mappings maintain exact fidelity to the original sources specified in the FRD. No intermediate sources, staging tables, or alternative sources have been introduced. The technical sources are direct representations or transformations of the original sources.

•

## ## 7. ETL MAPPING REQUIREMENTS

### ### 7.1 Source to Target Column Mapping

- Mapping 1: Customer Source to Validated Customer

|  | Source Column | Source Data Type | Target Column | Target Data Type | Transformation                  | Nullable |  |
|--|---------------|------------------|---------------|------------------|---------------------------------|----------|--|
|  | -----         | -----            | -----         | -----            | -----                           | -----    |  |
|  | CID           | Integer          | CID           | Integer          | Direct map (validation applied) | NOT NULL |  |
|  | FNAME         | String           | FNAME         | String           | Direct map (validation applied) | NOT NULL |  |
|  | LNAME         | String           | LNAME         | String           | Direct map (validation applied) | NOT NULL |  |

- Mapping 2: Order Source to Validated Order

|  | Source Column | Source Data Type | Target Column | Target Data Type | Transformation                  | Nullable |  |
|--|---------------|------------------|---------------|------------------|---------------------------------|----------|--|
|  | -----         | -----            | -----         | -----            | -----                           | -----    |  |
|  | OID           | Integer          | OID           | Integer          | Direct map (validation applied) | NOT NULL |  |
|  | CID           | Integer          | CID           | Integer          | Direct map (validation applied) | NOT NULL |  |
|  | ONAME         | String           | ONAME         | String           | Direct map (validation applied) | NOT NULL |  |

- Mapping 3: Validated Customer to Customer with FULL\_NAME

| Source Column | Source Data Type | Target Column | Target Data Type | Transformation                                    | Nullable |
|---------------|------------------|---------------|------------------|---------------------------------------------------|----------|
| -----         | -----            | -----         | -----            | -----                                             | -----    |
| CID           | Integer          | CID           | Integer          | Direct map                                        | NOT NULL |
| FNAME         | String           | FNAME         | String           | Direct map                                        | NOT NULL |
| LNAME         | String           | LNAME         | String           | Direct map                                        | NOT NULL |
| FNAME + LNAME | String           | FULL_NAME     | String           | initcap(trim(FNAME)) + ' ' + initcap(trim(LNAME)) | NOT NULL |

- Mapping 4: Customer + Order to Joined Data

|  | Source Column | Source Table | Source Data Type | Target Column | Target Data Type | Transformation            | Nullable |  |
|--|---------------|--------------|------------------|---------------|------------------|---------------------------|----------|--|
|  | -----         | -----        | -----            | -----         | -----            | -----                     | -----    |  |
|  | CID           | customers    | Integer          | CID           | Integer          | Direct map from customers | NOT NULL |  |
|  | FULL_NAME     | customers    | String           | FULL_NAME     | String           | Direct map from customers | NOT NULL |  |
|  | OID           | orders       | Integer          | OID           | Integer          | Direct map from orders    | NOT NULL |  |
|  | ONAME         | orders       | String           | ONAME         | String           | Direct map from orders    | NOT NULL |  |

- Mapping 5: Joined Data to Data with ORDER\_COUNT

| Source Column | Source Data Type | Target Column | Target Data Type | Transformation | Nullable |
|---------------|------------------|---------------|------------------|----------------|----------|
| -----         | -----            | -----         | -----            | -----          | -----    |
| CID           | Integer          | CID           | Integer          | Direct map     | NOT NULL |
| FULL_NAME     | String           | FULL_NAME     | String           | Direct map     | NOT NULL |
| OID           | Integer          | OID           | Integer          | Direct map     | NOT NULL |

|                  |         |             |         |                                    |          |
|------------------|---------|-------------|---------|------------------------------------|----------|
| ONAME            | String  | ONAME       | String  | Direct map                         | NOT NULL |
| OID (aggregated) | Integer | ORDER_COUNT | Integer | COUNT(OID) OVER (PARTITION BY CID) | NOT NULL |

- Mapping 6: Data with ORDER\_COUNT to Final Output